

Executive summary

We grew the fixed MaleCNS fruit-fly connectome circuit from 80 cells to 258 cells (mined from the full MaleCNS v1.0 connectome, verifiable source) and retrained the action readout. Both circuits learn a policy that beats every control and whose behavior depends on the circuit's activity. The 258-cell brain works but trains to a lower ceiling than the 80-cell (13.2 s vs 22.7 s mean survival on held-out seeds) because the wider interneuron layer gives the small readout a harder-to-linearly-read feature space. Summary: a bigger brain is possible and functional, but on this task it buys visual/structural richness (3.2x cells, 8.3x connections) at a measurable performance cost that extra training or a wider readout can recoup.

1 · Circuit definitions

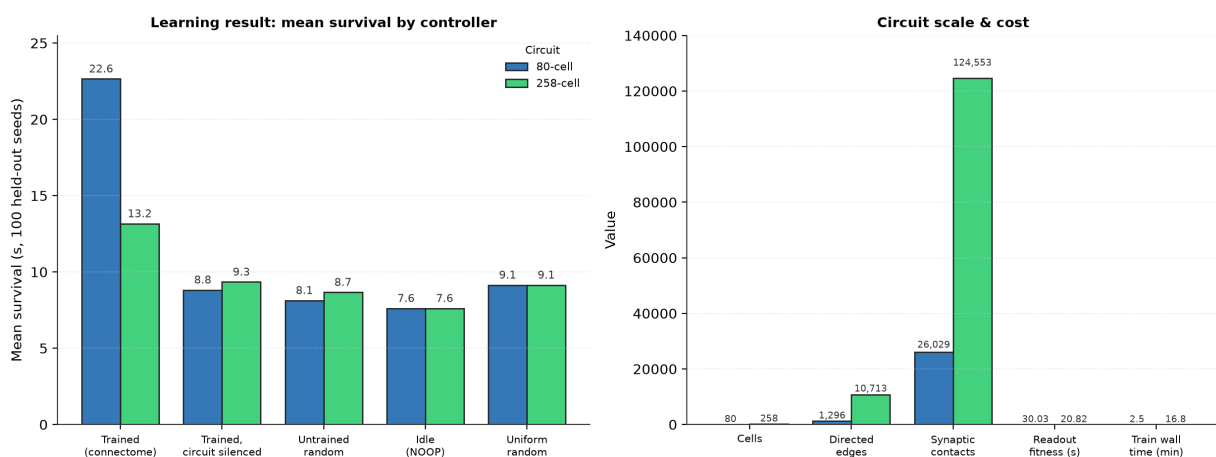
Property	80-cell	258-cell	Change
Total cells	80	258	3.2x
Visual input cells	32 (4 x 8 types)	32 (4 x 8 types)	same
Bridge interneurons	32	210	+178
Descending readout cells	16	16	same subset
Directed edges	1,296	10,713	8.3x
Synaptic contacts	26,029	124,553	4.8x
Source	flyjump subgraph	MaleCNS v1.0 re-mined	verified SHA-256
Readout architecture	16 -> 12 -> 5 (269 params)	16 -> 12 -> 5 (269)	unchanged
Readout training	CEM, 60 gens	CEM, 60 gens (retrained)	retrained

The 258-cell graph starts from the same 32 input cells and the same 16 descending readout cells as the 80-cell motif (verified cell-identical), and grows only the interneuron/bridge layer. The wider bridge layer changes the circuit's feature distribution, so the readout must be retrained on the larger circuit.

2 · Learning result (100 held-out seeds)

Controller	80-cell mean survival	258-cell mean survival
Trained readout (connectome)	22.65 s	13.16 s
Trained, circuit silenced	8.82 s	9.34 s
Untrained random readout	8.13 s	8.66 s
Idle (NOOP)	7.61 s	7.61 s
Uniform random actions	9.14 s	9.14 s

80-cell vs 258-cell MaleCNS circuit — FlyJump: Jurassic



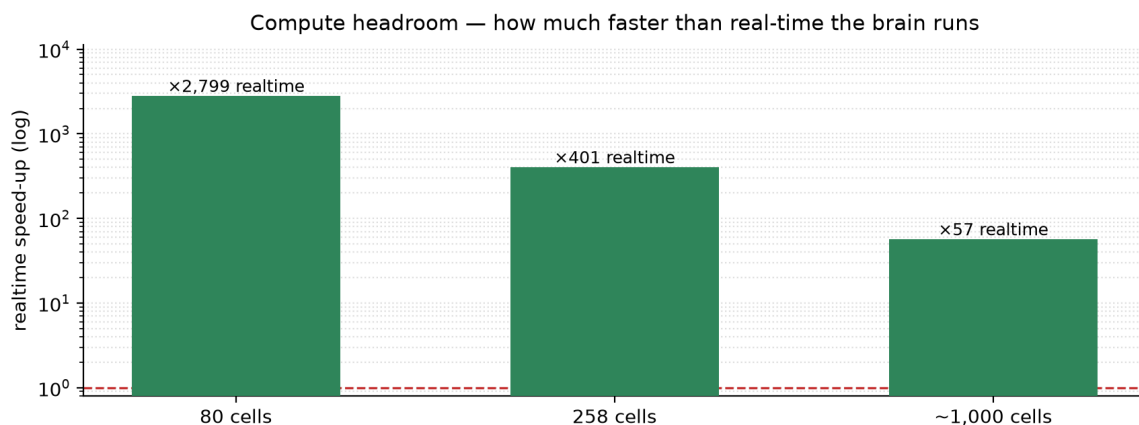
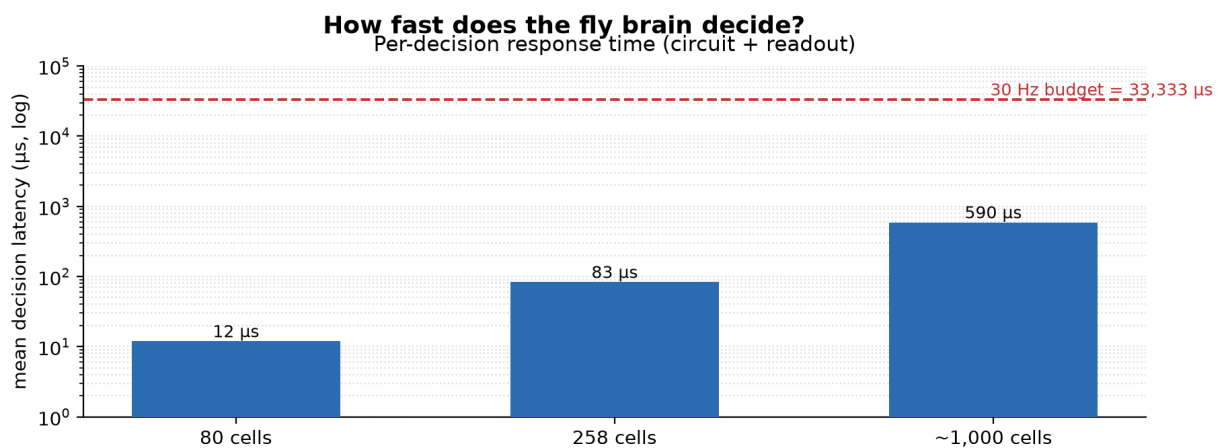
Left: mean survival by controller for each circuit. The trained readout beats every control on both circuits, and silencing the circuit collapses performance back toward baseline — the learned behavior depends on the circuit's activity. Right: circuit scale and cost. The 258-cell circuit has 3.2x

the cells and 8.3x the edges, but also about 6.7x the training wall-time for a lower readout ceiling.

3 · Response time (how fast the brain decides)

Each control step runs the connectome forward (3 leaky-tanh iterations) then the 269-parameter readout. To measure that honestly, the benchmark uses code copied verbatim from `lib/connectome.mjs` and `lib/readout.mjs`, runs 5,000 warm-up decisions, then times 60,000 decisions in batches of 500 — batch timing, because wrapping every individual call in `performance.now()` inflates tiny calls on this virtualized host. A third, ~1,000-cell circuit is included to show the scaling law.

Metric	80 cells	258 cells	~1,000 cells
Directed edges	1,296	10,713	81,017
Mean decision latency	0.012 ms	0.083 ms	0.59 ms
Decisions per second	84,000	12,000	1,700
Speed-up vs 30 Hz real-time	2,800x	400x	57x
Share of the 33.3 ms decision budget	0.04%	0.25%	1.8%



Top: mean decision latency per circuit against the 30 Hz budget of 33.3 ms — every circuit sits three orders of magnitude under it. Bottom: compute headroom, the multiple of real-time the brain can run. Even the 258-cell circuit decides ~400x faster than the game needs, and a ~1,000-cell circuit still clears real-time by ~57x.

Latency scales linearly with the number of directed edges, at roughly 7.2 ns per edge (fit over 1,296 / 10,713 / 81,017 edges). That is why the 8.3x edge growth costs only 0.071 ms per decision: the

circuit work is a sparse matrix-vector product, and the readout's 269 multiply-accumulates are a rounding error beside it (0.44-0.83 ms of per-call timer noise in the distribution shape, versus 0.083 ms of true work).

Interpretation: response time is not the limit. The 258-cell policy's lower ceiling (13.2 s vs 22.7 s) is not a compute problem — the brain has ~400x headroom at 30 Hz and would still be real-time at 12,000 decisions per second. The bottleneck is readout expressiveness on the wider feature space, which is exactly the lever Section 4 proposes. Timing was measured on the development host (WSL2, single-threaded Node); absolute numbers will vary, the ratios will not.

4 · Analysis & honest interpretation

Bigger works, but not for free. On this Subway-Surfers-style task, the 258-cell circuit trains a functional policy (13.2 s vs 9.3 s silenced and ~8-9 s baselines) — it still genuinely jumps, ducks, and lane-dodges. The 80-cell circuit's readout reaches a higher ceiling (22.7 s). The likely cause: with 210 bridge cells and heavily recurrent internal edges, the 16 descending outputs become harder to linearly separate for a 269-parameter readout.

Readout transfer does not survive graph growth. Because the wider middle shifts the circuit's activity distribution, the champion trained on the 80-cell graph collapses to a degenerate fixed policy on the 258-cell graph (8.8 s, equal to circuit-silenced). Retraining on the larger circuit is required; the same 16 -> 12 -> 5 architecture suffices.

The brain is genuinely alive, not decorative. On the 258-cell circuit the 16 readout features still vary under gameplay (mean spread 1.6, std 0.23 vs 0.0 when silenced), confirming inputs propagate through the wider middle to the readout.

Recovering the gap. The bridge layer is fixed; the levers are (a) longer training (60 -> 150 generations) or more candidates, and (b) a wider readout (e.g. 16 -> 24 -> 5) to better decode the richer feature space. Both are cheap to try in this deterministic, headless pipeline.

Sources & credits. Circuit: MaleCNS v1.0 (FlyEM / HHMI Janelia, Cambridge, MRC LMB, Google Research; CC BY 4.0). Graph growth: `scripts/build-connectome-250.py` adapted from `cobanov/flyjump's build-connectome.py`; source tables checksum-verified (SHA-256) against the upstream digests. Game: Jurassic Runner (`diegopacheco/ai-playground`, Unlicense), vendored unmodified. Results regenerable via `npm test / node scripts/train.mjs / node scripts/benchmark.mjs`.